

Introducción

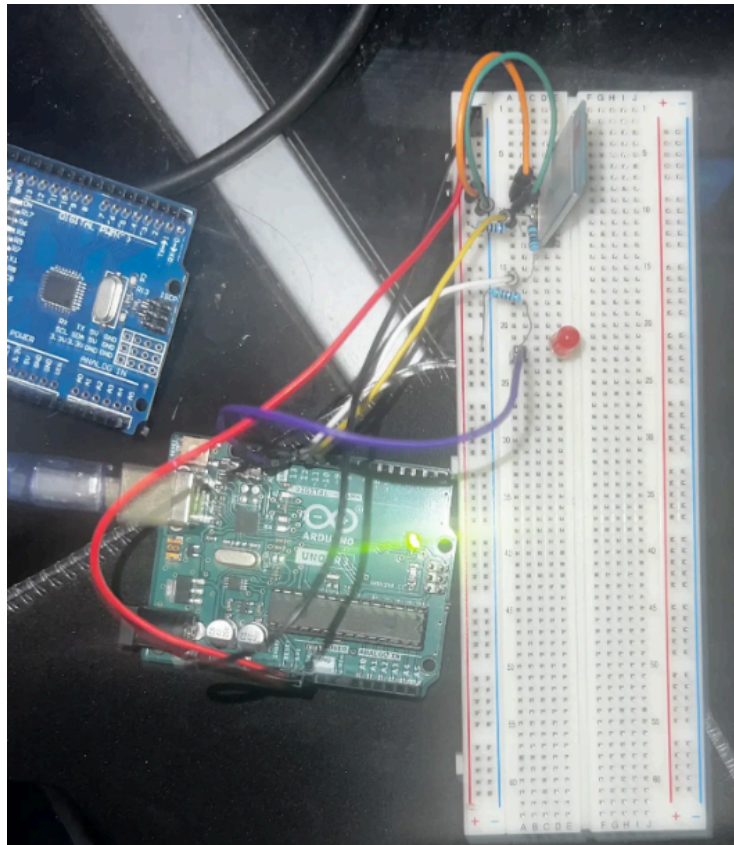
El objetivo de esta sesión fue crear un circuito controlado por un Arduino que pudiera ser operado de manera inalámbrica, utilizando otra computadora, por medio de un módulo de bluetooth HC05 y el programa CoolTerm. La funcionalidad final de esta práctica era simplemente prender y apagar un LED con la computadora conectada al módulo bluetooth.

Metodología

Materiales:

- Arduino.
- Protoboard.
- Módulo Bluetooth HC-05.
- LED Rojo.
- Cables jumper.
- Resistencias de 1000Ω , 2000Ω y 110Ω .
- Computadora host.
- Computadora cliente.

El arduino fue utilizado como el controlador principal del circuito, encargado de ejecutar el programa y gestionar la comunicación con el módulo Bluetooth. Por otro lado, el protoboard permitió posicionar y conectar de manera sencilla los diferentes componentes del proyecto. Antes de incorporar el LED, fue necesario realizar el setup inicial del módulo Bluetooth HC-05, configurándolo mediante comandos AT para establecer su funcionamiento. Durante esta etapa, se verificó la comunicación y se estableció el nombre del módulo como GORDOTOOTH. Asimismo, el módulo fue configurado como dispositivo esclavo, permitiendo que una computadora pudiera establecer la conexión inalámbrica con él. Una vez completada esta configuración, el HC-05 se regresó a su modo de emparejamiento para establecer comunicación inalámbrica con la computadora cliente. Finalmente, se incorporó un LED conectado al pin 13 del Arduino, el cual puede ser controlado mediante comandos enviados inalámbricamente desde la computadora, utilizando los valores 1 para encenderlo y 0 para apagarlo.



En el nuevo circuito, el Arduino se utilizó como controlador principal para establecer la comunicación inalámbrica con el módulo Bluetooth HC-05 y controlar el LED. El módulo Bluetooth fue conectado mediante comunicación serial utilizando los pines digitales 10 y 11 del Arduino. El pin 10 se utilizó como TX y el pin 11 como RX. Debido a que el nivel lógico de entrada del HC-05 es de aproximadamente 3.3 V, la conexión hacia su pin RX se realizó mediante un divisor de voltaje formado por dos resistencias (la resistencia de 1k Ω desde GND y la resistencia de 2k Ω desde el pin 11), reduciendo el voltaje proveniente del Arduino a un nivel adecuado para el módulo. Por otra parte, el pin 13 del Arduino se utilizó para controlar el LED, conectado en serie con una resistencia de 110 Ω para limitar la corriente y proteger el componente. Finalmente, se estableció una conexión común a tierra entre los componentes del circuito, permitiendo compartir una referencia eléctrica y garantizar el correcto funcionamiento de la comunicación y del LED.

Códigos utilizado

Setup.ino

```
1  #include <SoftwareSerial.h>
2
3  SoftwareSerial BT(10, 11);
4
5  void setup() {
6      Serial.begin(9600);
7      BT.begin(38400);
8      Serial.println("Enviar comandos AT: ");
9  }
10
11 void loop() {
12     if (Serial.available())
13         BT.write(Serial.read());
14
15     if (BT.available())
16         Serial.write(BT.read());
17 }
```

CódigoDeCiruito.ino

```
1  #include <SoftwareSerial.h>
2  SoftwareSerial BTSerial (10,11);
3  void setup() {
4      pinMode(13,OUTPUT);
5      Serial.begin(9600);
6      BTSerial.begin(9600);
7
8  }
9
10 void loop() {
11     while(BTSerial.available()>0){
12         char receivedChar = BTSerial.read();
13         Serial.print("Mensaje recibido: ");
14         Serial.println(receivedChar);
15         if(receivedChar == '1'){
16             Serial.println("Se recibio comando de encendido");
17             digitalWrite(13,HIGH);
18             BTSerial.println("Se encendio el LED");
19         }else if(receivedChar == '0'){
20             Serial.println("Se recibio comando de apagado");
21             digitalWrite(13,LOW);
22             BTSerial.println("Se apago el LED");
23         }else{
24             Serial.println("Se recibio comando invalido");
25             BTSerial.println("No hubo accion, comando invalido");
26         }
27     }
28
29 }
```

Explicación del código

Setup.ino

```
1  #include <SoftwareSerial.h>
2
3  SoftwareSerial BT(10, 11);
4
```

Primeramente se incluye la librería “SoftwareSerial”, la cual permite utilizar cualquier PIN digital como PIN serial (RX/TX). Después, crea uno de estos puertos utilizando el PIN 10 como receptor (RX) y el 11 como transmisor (TX). Esto es necesario para conectarse al Bluetooth.

```
5  void setup() {
6      Serial.begin(9600);
7      BT.begin(38400);
8      Serial.println("Enviar comandos AT: ");
9  }
10
```

Dentro del void setup, primero se inicia la comunicación serial con la computadora a la que está conectada a una velocidad de 9600 baudios (cambios de la señal por segundo). Esto se ocupa para poder ver mensajes en la terminal de la computadora. Después, inicia la comunicación serial con el módulo bluetooth a una velocidad de 38400 baudios. Por último, dentro del setup, se muestra el mensaje “Enviar comandos AT: “ en la terminal de la computadora.

```
11 void loop() {
12     if (Serial.available())
13         BT.write(Serial.read());
14
15     if (BT.available())
16         Serial.write(BT.read());
17 }
```

Dentro del void loop, se chequea constantemente si el usuario escribió un comando dentro del monitor serial. De ser este el caso, se lo manda al módulo Bluetooth para que este lo lea y pueda responder. El segundo *if* es justo para esto, revisando constantemente si el módulo ha producido una respuesta para mostrarla en el monitor serial. En esencia, es un bucle de comunicación entre el monitor y el módulo para mandar comandos y ver las respuestas que este produce.

CódigoDeCircuito.ino

```
1  #include <SoftwareSerial.h>
2  SoftwareSerial BTSerial (10,11);
```

Al igual que en el código pasado, se incluye la librería “SoftwareSerial” y se configuran los PINes 10 y 11 como el puerto serial.

```
3  void setup() {
4      pinMode(13,OUTPUT);
5      Serial.begin(9600);
6      BTSerial.begin(9600);
7  }
8
9
```

De nuevo, dentro del void setup se establece la comunicación con la computadora a una velocidad de 9600 baudios. Sin embargo, esta vez se establece la comunicación con el módulo bluetooth también a una velocidad de 9600 baudios. Esto se debe a que la velocidad de 38400 es para el modo AT (configuración) y 9600 es para modo de comunicación normal. Además de esto, también se establece el PIN 13 como una salida (para controlar el LED).

```
10 void loop() {
11     while(BTSerial.available()>0){
12         char receivedChar = BTSerial.read();
13         Serial.print("Mensaje recibido: ");
14         Serial.println(receivedChar);
15         if(receivedChar == '1'){
16             Serial.println("Se recibio comando de encendido");
17             digitalWrite(13,HIGH);
18             BTSerial.println("Se encendio el LED");
19         }else if(receivedChar == '0'){
20             Serial.println("Se recibio comando de apagado");
21             digitalWrite(13,LOW);
22             BTSerial.println("Se apago el LED");
23         }else{
24             Serial.println("Se recibio comando invalido");
25             BTSerial.println("No hubo accion, comando invalido");
26         }
27     }
28 }
29
```

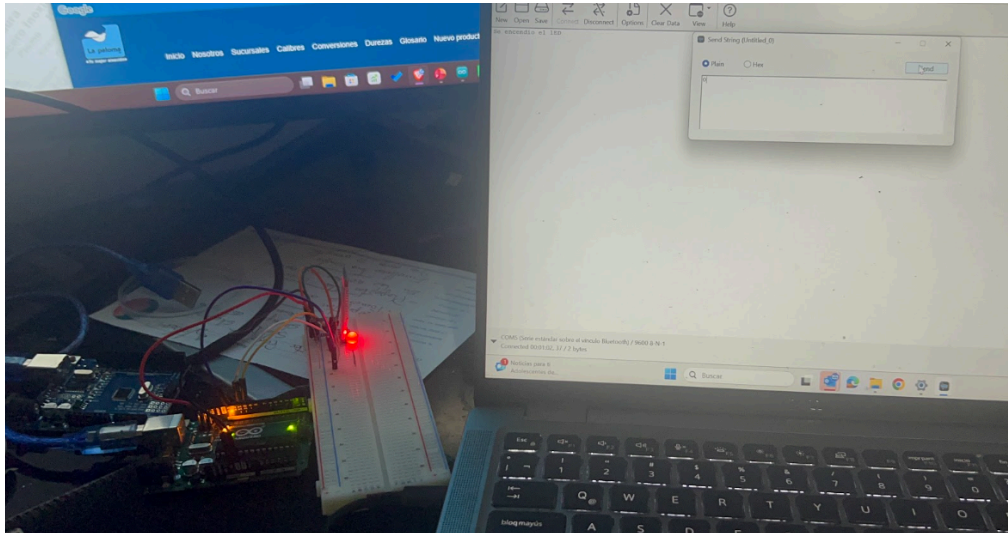
Dentro del void loop, se revisa constantemente si el módulo Bluetooth ha recibido un dato. Si detecta que se recibió algo, guarda el valor dentro de la variable *recievedChar*. Las variables tipo *char* solo pueden guardar un solo carácter. Una vez un dato recibido, el Arduino manda el mensaje “Mensaje recibido: ” junto el carácter a la terminal serial conectada. Después, en caso de que el carácter es 1, muestra el mensaje “Se recibio comando de encendido” en la terminal y prende el LED (y muestra un mensaje diciendo que se prendió el LED). Si recibe un 0, muestra el mensaje “Se recibio comando de apagado” en terminal y apaga el LED (y muestra un mensaje de que se apagó el LED). En

caso de recibir cualquier otro carácter, muestra el mensaje “Se recibió comando invalido” y le dice al dispositivo conectado por Bluetooth que no se tomó ninguna acción.

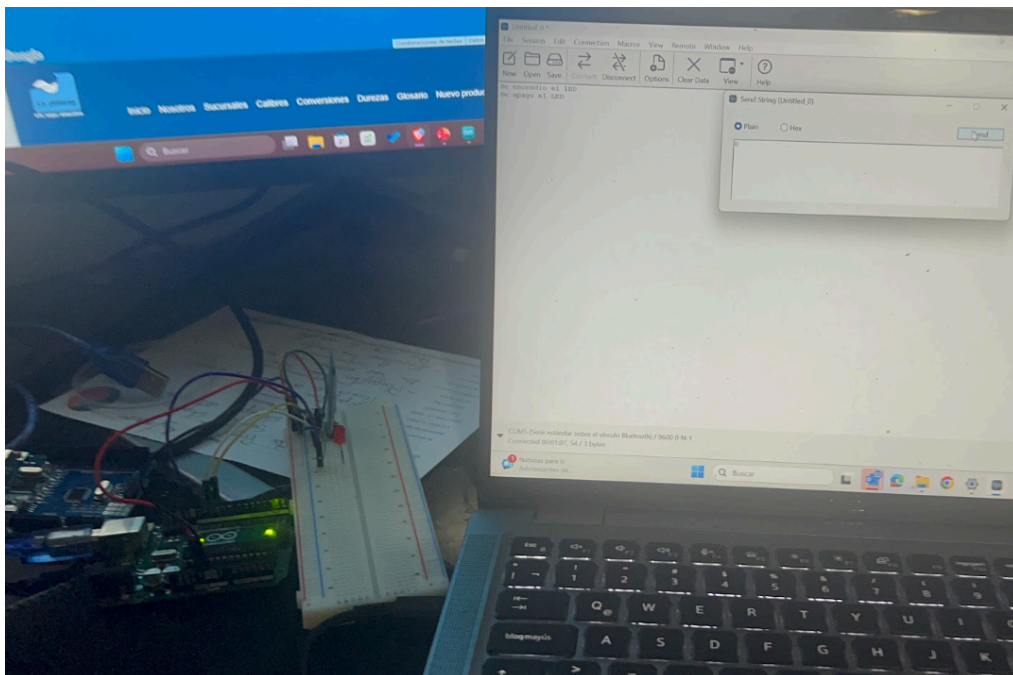
Resultados

Una vez el módulo HC-05 configurado, la computadora cliente conectada y el programa terminado, se lograron los resultados.

- Al ingresar 1, el LED prende. (En la terminal aparece 0, pero fue porque esta captura es posterior a apagar el LED).



- Después de ingresar el 0, el LED se ha apagado.



Conclusión

Los resultados obtenidos a partir del proyecto fueron los esperados. El módulo Bluetooth HC-05 logró establecer correctamente la comunicación inalámbrica con la computadora, permitiendo enviar comandos al Arduino para controlar el funcionamiento del LED. Durante el desarrollo fue necesario configurar previamente el módulo mediante comandos AT, establecer su nombre como GORDOTOOTH y configurarlo como dispositivo esclavo, lo que permitió realizar correctamente el emparejamiento con la computadora.

Por otro lado, las pruebas realizadas permitieron comprobar que el Arduino podía recibir los comandos enviados mediante Bluetooth y ejecutar la acción correspondiente. Al recibir el carácter '1', el LED conectado al pin 13 se encendió, mientras que al recibir '0', el LED se apagó. Esto permitió comprender la relación entre la comunicación inalámbrica, la programación del Arduino y el funcionamiento físico del circuito.

Por último, el proyecto permitió reforzar los conocimientos sobre comunicación serial y Bluetooth, así como sobre la configuración y conexión del módulo HC-05. Las pruebas realizadas también permitieron identificar la importancia de utilizar correctamente los niveles de voltaje, las conexiones RX y TX y la configuración de los parámetros de comunicación para obtener el funcionamiento esperado del circuito.